LimitRegressor: Converging is in fashion

Eduard Arzumanyan

December 28, 2025

Contents

1 Abstract **3**

2 Introduction **4**
2.1 How this type of model differs from others 4
2.2 Definitions and some properties 4

3 Related work **6**
3.1 Iterative refinement models 6
3.2 Fixed-point iteration, Lipschitz continuity, contraction mappings 7

4 The LimitRegressor Model **8**
4.1 Differentiation through the fixed-point 9
4.2 Unrolled Gradients Convergence 10
4.3 Training algorithm 13
4.4 Prediction process 14

5 Complexity analysis **15**
5.1 Setup and notations 15
5.2 The complexity of inference (prediction) 16
5.3 The complexity of training process 17

6 Pseudocodes of LimitRegressor **18**
6.1 Prediction via Fixed-Point Iteration 18
6.2 Training via Implicit Differentiation 18
6.3 Unrolled Optimization as a Baseline 20

1 Abstract

We propose *LimitRegressor*, a regression model whose output is defined implicitly as the fixed point of a contraction mapping. Given an input $x \in \mathbb{R}^d$, the model produces predictions by iteratively refining an estimate according to

$$y_{k+1} = y_k + \alpha\, \phi(F_\theta(x) - y_k)\,,$$

where F_θ is a parametric function and ϕ is a Lipschitz continuous nonlinearity. Under mild conditions on the step size α, the iteration converges to a unique fixed point y^*, which constitutes the model output.

Unlike standard feed-forward regressors, LimitRegressor explicitly enforces convergence and allows a controllable trade-off between computational cost and prediction accuracy. We derive a closed-form expression for the gradient of the loss with respect to model parameters using implicit differentiation, showing that gradients can be computed without backpropagating through the iterative process. This yields training with constant memory complexity, independent of the number of refinement steps.

We establish theoretical guarantees for existence, uniqueness, and convergence of the fixed point, analyze the computational complexity of both inference and training, and compare the proposed approach with unrolled optimization and Deep Equilibrium Models. The resulting framework provides a simple yet mathematically grounded method for fixed-point regression with predictable behavior and efficient training.

2 Introduction

For the beginning it is very important to give some basic but very important definitions, formulations before proceeding to model. But before that let's talk about differences of LimitRegressor and other popular models.

2.1 How this type of model differs from others

Unlike other models, LimitRegressor is implicitly defined as fixed point $y^* = lim_{k\to\infty} y_k$. This model is iterative, so it can refine predictions progressively but also heavily depends on previous output y_{k-1}. Model is unique by its structure because it contains lots of concepts from Calculus. This model has similarities with RNN (Recurrent Neural Network) - both iterative,but RNN - over time steps, LimitRegressor - per input to compute a fixed point. The closest concept to LimitRegressor are Deep Equilibrum models - they define outputs as the fixed point of a parametric mapping, but LimitRegressor is better because of its explicit α-step update with a Lipschitz ϕ, giving a simpler and controllable convergence guarantee. The contraction property ensures almost unique outputs and stable convergence. We can also trade computation versus accuracy via ϵ and k_{max}, standard networks this iterative refinement.

2.2 Definitions and some properties

Let $f : \mathbb{N} \to \mathbb{R}$. From now on $y_n := f(n)$ and let's call it sequence. Let's say sequence y_n converges to $A \in \mathbb{R}$ if:

$$\forall \epsilon > 0 \ \exists N \in \mathbb{N} \ s.t. (n \geq N \Rightarrow |y_n - A| < \epsilon)$$

Meaning of this beauty is:

- ϵ represents how close you want the sequence to be to the limit L
- N tells you how far you must go into the sequence so that from that point onward, the distance between a_n and L is less then ϵ.
- The condition must work for every positive ϵ, especially for extremely small ones

Let $f : X \to \mathbb{R}$. f is continuous at a point $a \in X$ if:

$$\forall \epsilon > 0 \ \exists \delta > 0 \ s.t. \ \forall x \in X \ (|x - a| < \delta \Rightarrow |f(x) - f(a)| < \epsilon)$$

More simply, function f is continuous at a point $a \in X$ if these 3 conditions are met:

1. **$f(a)$ is defined**
2. **the limit of $f(x)$ as x approaches a exists**
3. $\lim_{x \to a} f(x) = f(a)$

Let $f : X \to \mathbb{R}$. f is differentiable at a point $a \in X$, if X contains an open interval containing a and the limit

$$L = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

or it is the same as:

$$f'(a) = \lim_{x \to a} \frac{f(x) - f(a)}{x - a}$$

exists. Let's prove a simple theorem from Calculus.

Theorem. Let $f : \mathbb{R} \to \mathbb{R}$ be a function and let $x_0 \in \mathbb{R}$. If f is differentiable at x_0 , that is, the limit

$$\lim_{x \to x_0} \frac{f(x) - f(x_0)}{x - x_0}$$

exists and is finite, then f is continuous at x_0.

Proof. From the definition

$$\frac{f(x) - f(x_0)}{x - x_0} \to f'(x_0) \text{ as } x \to x_0$$

. Let's multiply each side by $x - x_0$:

$$f(x) - f(x_0) = (x - x_0)\frac{f(x) - f(x_0)}{x - x_0}$$

. Let's take the limit of both sides:

$$\lim_{x \to x_0} [f(x) - f(x_0)] = \lim_{x \to x_0} (x - x_0)\frac{f(x) - f(x_0)}{x - x_0}$$

The right side is equal to $0f'(x_0) = 0$. Therefore:

$$\lim_{x \to x_0} f(x) = f(x_0)$$

which is exactly the definition of continuity at x_0 $\square$

3 Related work

Before proceeding to model it is important also to talk about very important concepts to understand the intuition, logic behind LimitRegressor.

3.1 Iterative refinement models

An iterative refinement model is a machine learning model that improves its predictions progressively through repeated application of an update mapping:

$$y_{k+1} = y_k + \Delta(y_k, x, \theta), \quad k \in \mathbb{N}$$

where:

- y_k is prediction at iteration k
- x is the input
- θ are learnable parameters
- Δ is refinement function that adjusts the current estimate

As already talked, the process here continues "until convergence" or stopping criterion is reached:

$$||y_{k+1} - y_k|| < \epsilon \quad \text{or} \quad k = k_{max}$$

Standard feed-forward models make predictions in one pass, which is limitation to model. Such type of models iteratively improve their own predictions. if $\Delta(y, x, \theta)$ is Lipschitz with Constant $L < 1$, then the mapping $y \to y + \Delta(y, x, \theta)$ is a contraction. As mentioned, this ensures unique convergence to a fixed point y^*.

3.2 Fixed-point iteration, Lipschitz continuity, contraction mappings

A function $\phi : \mathbb{R}^n \to \mathbb{R}^n$ is called Lipschitz continuous if there exists $L \geq 0$ such that:

$$||\phi(y_1) - \phi(y_2)|| \leq L||y_1 - y_2|| \quad \forall y_1, y_2$$

If $L < 1$, then ϕ is a contraction mapping, ensuring unique fixed points and convergence as proved. Let $F : \mathbb{R}^n \to \mathbb{R}^n$ function. A point y^* is called fixed point if:

$$F(y^*) = y^*$$

Fixed point iteration (FPI) is a method to compute y^* by repeatedly applying F starting from an initial guess y_0:

$$y_{k+1} = F(y_k), \quad k \in \mathbb{N}$$

A function F is a contraction if there exists $L \in [0; 1)$ such that:

$$||F(y_1) - F(y_2)|| \leq L||y_1 - y_2 \quad \forall y_1,\ y_2 \in \mathbb{R}^n.$$

Theorem. If $F : X \to \mathbb{R}$ is a contraction, then it has a unique point y^*, and the sequence

$$y_{k+1} = F(y_k)$$

converges to y^* for any initial y_0

Proof. Let's show that sequence y_k is convergent. Let $y_0 \in X$ be arbitrary and define the iterative sequence:

$$y_{k+1} = F(y_k), \quad k \in \mathbb{N}$$

$$||y_{n+p} - y_n|| \leq \sum_{k=0}^{p-1} ||y_{n+k+1} - y_{n+k}|| \leq \sum_{k=0}^{p-1} L^{n+k}||y_1 - y_0|| = L^n \frac{1 - L^p}{1 - L}||y_1 - y_0||$$

Since $L \in [0; 1)$, as $n \to \infty$, $L^n \to 0$. Hence, y_k is Cauchy, and therefore convergent.

$$\lim_{k\to\infty} y_k = y^*$$

By continuity of F

$$F(y^*) = F(\lim_{k\to\infty} y_k) = \lim_{k\to\infty} F(y_k) = \lim_{k\to\infty} y_{k+1} = y^*$$

So, y^* is a fixed point.

Suppose there exists another fixed point $z^* \neq y^*$. Then:

$$||y^* - z^*|| = ||F(y^*) - F(z^*)|| \leq L||y^* - z^*||$$

Since $L < 1 \Rightarrow y^* = z^*$, which is contradiction. Hence, y^* is unique $\square$

4 The LimitRegressor Model

Here we go, we are fully ready to define our model and all algorithms related to it. Let's start!

Let y_k be a sequence defined by this formula:

$$y_{k+1} = y_k + \alpha\phi(F_\theta(x) - y_k), \quad y_0 = 0$$

These parameters will be use across this section for defining,understanding LimitRegressor.

- ϵ - positive arbitrary small real number for regulating the convergence of prediction
- δ - positive arbitrary small real number for regulating the convergence of loss
- α - positive step-size parameter controlling iteration dynamics
- $\phi(\cdot)$ - Lipschitz continuous function
- k_{max} - prediction maximum iteration count
- q_{max} - maximum epochs count in training
- η - learning rate

If ϕ is not Lipschitz function the convergence is not guaranteed.

4.1 Differentiation through the fixed-point

The heart of training of the LimitRegressor lies a beautiful and useful mathematical idea which simplifies and solves the problem of computation of gradient of Loss function - differentiation through the fixed-point.

Theorem

$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$

Proof. At convergence we have

$$\begin{aligned} y^* \approx y^* + \alpha\phi(F_\theta(x) - y^*) \Rightarrow \phi(F_\theta(x) - y^*) = 0 \\ G(y^*, \theta) := \phi(F_\theta(x) - y^*), \ \frac{dG(y^*, \theta)}{d\theta} = 0 \\ \frac{\partial G}{\partial y^*} \cdot \frac{\partial y^*}{\partial \theta} + \frac{\partial G}{\partial \theta} = 0 \end{aligned}$$

From here we have:

$$\frac{\partial y^*}{\partial \theta} = -(\frac{\partial G}{\partial y^*})^{-1} \cdot \frac{\partial G}{\partial \theta}$$

From the other side we have:

$$\frac{\partial G}{\partial y^*} = -\phi'(F_\theta(x) - y^*)$$

$$\frac{\partial G}{\partial \theta} = \phi'(F_\theta(x) - y^*) \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$

Substitute back all these:

$$\frac{\partial y^*}{\partial \theta} = -(-\phi'(F_\theta(x) - y^*))^{-1} \cdot (\phi'(F_\theta(x) - y^*) \cdot \frac{\partial F_\theta(x)}{\partial \theta})$$

$$\frac{\partial y^*}{\partial \theta} = \frac{\partial F_\theta(x)}{\partial \theta}$$

Hence:

$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial y^*}{\partial \theta} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$

□

4.2 Unrolled Gradients Convergence

Now, let's prove a beautiful fact on which the convergence of LimitRegressor heavily relies on.

Theorem. Consider the iterative formula

$$y_{k+1} = y_k + \alpha\phi(F_\theta(x) - y_k), \quad y_0 = 0$$

where:

- $F_\theta : \mathbb{R}^d \to \mathbb{R}^m$ is continuously differentiable in θ,
- $\phi : \mathbb{R}^m \to \mathbb{R}^m$ is continuously differentiable,
- ϕ is Lipschitz with constant L_ϕ,
- $0 < \alpha L_\phi < 1$

Let $y^* = \lim_{k\to\infty} y_k$, and define the loss

$$L(\theta) = l(y^*, t),$$

with l continuously differentiable. Let $L_k(\theta) := l(y_K, t)$ denote the loss obtained after K unrolled iterations.

Then:

$$\lim_{K\to\infty} \nabla_\theta \mathcal{L}_K(\theta) = \nabla_\theta \mathcal{L}(\theta) = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$

Proof. Let's define mapping T, which is contraction by assumption:

$$T(y) := y + \alpha\phi(F_\theta(x) - y)$$

Hence:

$$||T(y_2) - T(y_1)|| \leq \alpha L_\phi ||y_1 - y_2||$$

We've already proven that:

$$y_k \to y^*$$

$\phi$is differentiable, therefore the Jacobian of T w.r.t y exists:

$$\frac{\partial T}{\partial y}(y) = I - \alpha\phi'(F_\theta(x) - y)$$

At the fixed point it equals

$$J := I - \alpha\phi'(F_\theta(x) - y^*)$$

And because $||\phi'|| \leq L_\phi$,

$$||J|| \leq \alpha L_\phi < 1$$

Therefore J is a strict contraction.

Define:

$$g_k := \frac{\partial y_k}{\partial \theta}$$

Also we have:

$$y_{k+1} = T(y_k)$$

And for g_{k+1} we will have:

$$g_{k+1} = \frac{\partial y_{k+1}}{\partial \theta} = \frac{\partial T}{\partial y}(y_k) g_k + \frac{\partial T}{\partial \theta}(y_k)$$

$$\frac{\partial T}{\partial \theta}(y_k) = \alpha \phi'(F_\theta(x) - y) \frac{\partial F_\theta(x)}{\partial \theta}$$

Define:

$$A_k := I - \alpha \phi'(F_\theta(x) - y_k), \quad b_k := \alpha \phi'(F_\theta(x) - y_k) \frac{\partial F_\theta(x)}{\partial \theta}$$

And from here we have:

$$g_{k+1} = A_k g_k + b_k$$

It is obvious that as $k \to \infty$,

$$A_k \to J, \quad b_k \to b := \alpha \phi'(F_\theta(x) - y^*) \frac{\partial F_\theta(x)}{\partial \theta}$$

Since $||A_k|| \leq \alpha L_\phi < 1$ for uniformly large k, g_k converges:

$$\lim_{k \to \infty} g_k =: g^*$$

And g^* satisfies:

$$g^* = J g^* + b$$

Let's solve this equality:

$$(I - J) g^* = b g^* = \frac{\partial F_\theta(x)}{\partial \theta}$$

By the chain rule we have:

$$\nabla_\theta \mathcal{L}_K = \frac{\partial l}{\partial y_K} \cdot g_K$$

As $K \to \infty$:

$$y_K \to y^*, \quad g_K \to g^*$$

Therefore:

$$\frac{\partial l}{\partial y_K} \cdot g_K \to \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta} = \nabla_\theta \mathcal{L}$$

and the proof is complete. $\square$

4.3 Training algorithm

First we must choose suitable loss function depending on the problem we want to solve. The general formula for that is:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} l(y^*(x_i), t_i)$$

where

- N is the count of rows in the given dataset
- x_i is the i-th data point
- t_i is the i-th target, the right answer

Now the training algorithm begins and is done this way:

1. Choose random θ
2. Compute $y^*(x)$
3. Compute $\frac{\partial l}{\partial y^*}$
4. Using the already proved formula for differentiation through the fixed-point compute
$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$
5. $\theta := \theta - \eta \nabla_\theta \mathcal{L}$

The training algorithm ends until one of these is reached:

1. $|y_{k+1} - y_k| < \epsilon$
   $k = k_{max}$
2. $|\mathcal{L}^{(t+1)} - \mathcal{L}^{(t)}| < \delta$
   $q = q_{max}$
   Early stopping rounds are met

4.4 Prediction process

$$y_{k+1} = y_k + \alpha \cdot \phi(F_\theta(x) - y_k),\ y_0 = 0 \quad (1)$$

The prediction result is equal to:

$$y^* = \lim_{k \to \infty} y_k$$

we cannot compute this directly using modern technologies but to get approximate value is still possible. It will be done this way:

1. Choose arbitrary small $\epsilon > 0$
2. Repeat iteration (1) until $||y_{k+1} - y_k|| < \epsilon$ or until $k = k_{max}$

Here it is the right prediction - our model not just predicted a value - it converged to the result and that is great and beautiful.

5 Complexity analysis

5.1 Setup and notations

Once again, for clarity let's consider these notations:

- $x \in \mathbb{R}^d$ - input
- $y \in \mathbb{R}^m$ - output
- $F_\theta : \mathbb{R}^d \to \mathbb{R}^m$ - neural network
- $\phi : \mathbb{R}^m \to \mathbb{R}^m$ - Lipschitz function
- k - number of iterations until convergence
- k_{max} - maximum allowed iterations count
- C_F - cost of forward pass F_θ
- C_ϕ - cost of applying ϕ

We assume here that ϕ is elementwise or linear-time, norm computation is $\mathcal{O}(m)$, therefore

$$C_\phi = \mathcal{O}(m)$$

5.2 The complexity of inference (prediction)

Each iteration computes:

$$y_{k+1} = y_k + \alpha \cdot \phi(F_\theta(x) - y_k)$$

This requires:

- Compute $F_\theta(x) : \mathcal{O}(C_F)$
- Vector subtraction: $\mathcal{O}(m)$
- Applying ϕ: $\mathcal{O}(m)$
- Updating y_k: $\mathcal{O}(m)$

If convergence happens in k iterations, the complexity is:

$$\mathcal{O}(k(C_F + m))$$

In worst case it happens in k_{max} iterations and complexity is:

$$\mathcal{O}(k_{max}(C_F + m))$$

From the other side we have that $T(y) = y + \alpha \cdot \phi(F_\theta(x) - y)$ is a contraction mapping witch factor $\alpha L_\phi < 1$:

$$||y_k - y|| \leq (\alpha L_\phi)^k ||y_0 - y||$$

Therefore:

$$k = \mathcal{O}(\log \frac{1}{\epsilon})$$

Hence, the inference complexity is:

$$\mathcal{O}((C_F + m) \log \frac{1}{\epsilon})$$

Memory complexity of this process consists of:

- Store y_k: $\mathcal{O}(m)$
- Store model parameters: $\mathcal{O}(|\theta|)$

Hence, it is equal to:

$$\mathcal{O}(m + |\theta|)$$

5.3 The complexity of training process

It is obvious that complexity of forward phase is the same as inference:

$$\mathcal{O}(k(C_F + m))$$

We have already proved that,

$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta}{\partial \theta}$$

and the gradient computation requires one backward pass through F_θ and its cost is $\mathcal{O}(C_f)$.
Hence, total cost per sample is:

$$\mathcal{O}(k(C_F + m) + C_F)$$

Memory complexity of training process is equal to the inference process memory complexity.

6 Pseudocodes of LimitRegressor

6.1 Prediction via Fixed-Point Iteration

Given an input $x \in \mathbb{R}^d$, the prediction of the LimitRegressor is defined implicitly as the fixed point of an iterative refinement process. Starting from an initial estimate $y_0 = 0$, the model repeatedly applies a contraction mapping until convergence.

The full prediction procedure is summarized in Algorithm 1. At each iteration, the current estimate is updated by moving in the direction prescribed by the Lipschitz function ϕ, applied to the residual $F_\theta(x) - y_k$. Convergence is detected once the change between successive iterates falls below a prescribed tolerance ε, or when the maximum number of iterations $k_{\max}$ is reached.

Under the assumption $0 < \alpha L_\phi < 1$, the update mapping is a contraction and the sequence $\{y_k\}_{k \geq 0}$ converges to a unique fixed point y^*, independent of initialization.

Algorithm 1 LimitRegressor Prediction

Require: Input $x \in \mathbb{R}^d$, parameters θ
Require: Step size $\alpha > 0$, tolerance $\varepsilon > 0$, maximum iterations $k_{\max}$

```
Initialize y_0 ← 0
for k = 0 to k_max − 1 do
    y_{k+1} ← y_k + α φ(F_θ(x) − y_k)
    if ‖y_{k+1} − y_k‖ < ε then
        break
    end if
end for
return y* ← y_{k+1}
```

6.2 Training via Implicit Differentiation

Training the LimitRegressor requires computing gradients of the loss with respect to model parameters, despite the output being defined implicitly as a fixed point. Rather than backpropagating through all refinement steps, we leverage implicit differentiation at convergence.

The resulting training procedure is presented in Algorithm 2. For each input x, the fixed-point prediction $y^*(x)$ is first computed using Algorithm 1. The loss $l(y^*, t)$ is then evaluated, and gradients are obtained using the closed-form expression

$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta},$$

which follows directly from the fixed-point condition $\phi(F_\theta(x) - y^*) = 0$.

This approach yields gradients identical to those obtained by infinite unrolling, while requiring only a single backward pass through F_θ. As a result, the memory complexity of training is independent of the number of refinement iterations.

Algorithm 2 LimitRegressor Training via Implicit Differentiation

Require: Dataset $\{(x_i, t_i)\}_{i=1}^N$
Require: Learning rate $\eta > 0$
Initialize parameters θ randomly
repeat
  for each mini-batch $\mathcal{B}$ **do**
    for each $(x, t) \in \mathcal{B}$ **do**
      Compute $y^*(x)$ using Algorithm 1
      Compute loss $l(y^*, t)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{|\mathcal{B}|} \sum l(y^*, t)$
    Compute gradient:

$$\nabla_\theta \mathcal{L} = \frac{\partial l}{\partial y^*} \cdot \frac{\partial F_\theta(x)}{\partial \theta}$$

    Update parameters: $\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}$
  end for
until stopping criterion is met

6.3 Unrolled Optimization as a Baseline

For completeness, one may alternatively train the model by unrolling the refinement process for a fixed number of iterations and applying standard backpropagation through time. This procedure is outlined in Algorithm 3.

While this approach yields correct gradients in the limit of infinite unrolling, it incurs a computational cost that grows linearly with the number of iterations. In particular, both memory usage and backward-pass complexity scale as $\mathcal{O}(K)$, rendering this method impractical for large K.

In contrast, the implicit differentiation approach adopted in this work achieves the same gradients at convergence while maintaining constant memory complexity.

Algorithm 3 LimitRegressor Training via Unrolled Backpropagation

Require: Dataset $\{(x_i, t_i)\}$
Require: Learning rate η, unroll depth K
Initialize parameters θ
for each mini-batch $\mathcal{B}$ **do**
    for each $(x, t) \in \mathcal{B}$ **do**
        $y_0 \leftarrow 0$
        for $k = 0$ to $K - 1$ **do**
            $y_{k+1} \leftarrow y_k + \alpha\phi(F_\theta(x) - y_k)$
        end for
        Compute loss $l(y_K, t)$
    end for
    Backpropagate through all K iterations
    Update parameters θ
end for